



Relatório: Experimento Bônus - Sistema de Detecção Automática de Congestionamento

1. Objetivo

Este experimento extra teve como objetivo validar a capacidade analítica da ferramenta *TCP Congestion Observatory (TCP-CO)* através da implementação de um script Python em *user-space*. O script foi desenhado para atuar como um "radar" de anomalias, varrendo o DataFrame gerado pelo eBPF e emitindo alertas visuais automáticos sempre que uma condição crítica de congestionamento fosse detectada.

A regra matemática definida para o alerta crítico foi:

1. Redução severa da Janela de Congestionamento: $Cwnd_{atual} \leq 0.5 \times Cwnd_{anterior}$ (Queda $\geq 50\%$).
2. Aumento simultâneo de falhas: $Retrans_{atual} > Retrans_{anterior}$.

2. Metodologia

Para forçar o acionamento do alarme, emulou-se um cenário de rede degradada com 1% de perda de pacotes (`tc netem loss 1%`) na interface de `loopback`. O tráfego foi gerado via `iperf3` durante 30 segundos. O teste foi executado duas vezes: uma utilizando o algoritmo TCP RENO e outra utilizando o TCP CUBIC, a fim de avaliar a sensibilidade do script aos diferentes mecanismos de redução.

3. Análise dos Resultados (Alertas Disparados)

A execução do script sobre os dados coletados comprovou o pleno funcionamento da lógica de detecção, extraindo inteligência operacional pura a partir da telemetria bruta do kernel.

3.1. O Comportamento Esperado (TCP RENO)

Como demonstrado no *log* de alertas do TCP RENO (Ref: `Alerta_Reno.png`), a mecânica do algoritmo clássico validou a regra perfeitamente. O RENO utiliza a política *Multiplicative Decrease*, cortando a janela rigorosamente pela metade perante a detecção de uma perda.

O script capturou esta assinatura matemática em milissegundos exatos. Por exemplo, às `15:28:42.353` , o alarme disparou indicando:

 *Cwnd caiu de 6 para 3 pacotes (Redução de 50.0%)*

3.2. A Surpresa Prática vs. Teoria (TCP CUBIC)

O teste com o TCP CUBIC (Ref: `Alerta_CUBIC.png`) trouxe uma observação empírica riquíssima para o laboratório. Teoricamente, o CUBIC aplica um fator de redução multiplicativa de `0.7`, o que resultaria numa queda de apenas `30%`, não sendo suficiente para acionar a regra do alarme ($\geq 50\%$).

Contudo, a taxa de 1% de perda contínua cria um ambiente de altíssima agressividade. Os dados provam que, sob estresse severo, o CUBIC sofre múltiplas perdas na mesma janela de transmissão ou estouros de temporizador (*Retransmission Timeout - RTO*), forçando a janela a colapsar drasticamente para proteger a rede, sobrepondo a regra teórica dos `30%`. O script detectou com precisão essas anomalias críticas, emitindo alertas como:

 Cwnd caiu de 11 para 2 pacotes (Redução de 81.8%) às 15:24:34.711 .

4. Conclusão

O desenvolvimento e a execução deste script de alerta comprovam que a observabilidade baseada em eBPF transcende a mera plotagem de gráficos. Ela permite a criação de sistemas de *Threshold Alerting* (Alertas baseados em Limiares) em tempo real, sem onerar a CPU ou modificar o sistema operativo. Em um ambiente de Data Center ou infraestrutura em nuvem, esta exata lógica poderia ser acoplada a um sistema de *pager* (como o Prometheus/Alertmanager) para notificar as equipas de engenharia no exato milissegundo em que uma rota de rede entrasse em colapso.